

Work Assignment 2

Context

The main goal of this assignment is to develop in students transversal skills applied to a specific topic in the Curricular Unit *Advanced Computing Architectures*: the methodology for characterising performance bottlenecks on a computing platform and/or code profiling and its performance analysis on that platform. These skills will be developed through training in literature search, reading & interpreting scientific papers, planning experimental work, synthesising relevant information, writing a short essay on a given theme and a short (10 min) oral communication and discussion of the results.

You should start by reading the documentation on the performance model Roofline (already supplied), the Amdahl law applied to heterogeneous multicore platforms and the respective Gustafson extension to this law.

The use of Intel VTune Profiler, Intel Advisor, Nvidia Nsight (*nsys*), performance counters (through *perf*), or other profilers available at Deucalion is recommended when it may enhance the performance analysis.

This work should be performed by a 3-student team, who will deliver two reports and a single oral presentation.

The code base for this assignment is available in Deucalion at:

`/projects/F202500010HPCVLABUMINHO/ACA/WA2`

Task 1

Full characterisation of the hardware platform

1.1 Fully characterise the CPU and GPU devices in each Deucalion partition (x86, ARM, and A100):

- (i) Manufacturer, model, CPU-device manufacturer/model/reference, main memory latency and size;
- (ii) For the CPU and GPU devices, give more details on #cores and peak FP performance;
- (iii) For the memory hierarchy, present cache details and the memory access bandwidth (from the LLC on chip).

Explain how you got these figures.

1.2 Build and compare the roofline model for single-precision FP on two computer systems: the AMD 7742 of the x86 partition and the A64FX of the ARM partition.

Show the roofline for both systems in the same operational-intensity graph; follow the suggestions in Appendix A of the 2008 paper on Roofline. It is advised to study labs 2 and 2.5 to perform this analysis.

1.3 Add ceilings to the rooflines, as suggested in the paper, and clearly justify each ceiling in the context of the algorithm to be tested. **Order** these ceilings accordingly.

1.4 Prepare the testing methodology based on the goals for the analyses to be performed in Task 2. This should consider **everything necessary to replicate your measurements:**

- The description of the testing protocol
 - The software and hardware of the target platform, the number of measurements to be performed, and how the statistical analysis will be performed
 - Profilers used, along with a justification of why those were chosen
- The list of tests to be performed
 - I.e., scalability analysis on hardware X with up to N threads, parallelisation overhead measurements, miss rates, ... (can be extended for the final report)
 - List of metrics and their purpose (time, energy, performance counters, ...)
 - Justification for each analysis: goals, what they will enable to understand about the software/hardware, and expected outcomes
- To analyse the code execution, **select (and justify)** the sizes for 3 different data structure(s):
 - That will completely fit in the L1 cache,
 - That only requires access to LLC cache,
 - That requires access to external RAM (> 1GiB).

It is acceptable to have deviations from the testing protocol in the final report, such as adding or removing tests that were not originally planned. However, these have to be justified in the final report.

Deliverable (by December 5th, 23:59): 2-page report, following the same template as in Task 3, describing the work performed in tasks 1.1 to 1.4.

Task 2

Performance of 3D Jacobi algorithm

2.1 Identify all performance counters available for the systems in each partition. From these, **select (and justify)** the most relevant ones to plot the arithmetic intensity of the algorithms in the rooflines developed in Task 1.

2.2 Build tables for the measurements planned in 1.4 of the incremental implementations performed. Keep track of the differences among optimisations. These tables should be used as the basis for the plots to be presented in Task 3.

2.3 Write an optimised version of the algorithm provided in this WA's repository. Consider vectorising the code, changing the data structures and access patterns to improve locality, and parallelising the code with OpenMP or MPI+OpenMP (if it makes sense given the architecture of the target CPUs). Note that these optimisations are important to this WA grade, but analysing the algorithm's behaviour on different hardware is more important. Manage your time wisely between optimising and assessing the code.

2.4 To analyse the code execution, **select** (and justify) the sizes for 3 different data structure(s):

- (i) That will completely fit in the L1 cache,
- (ii) That only requires access to LLC cache,
- (iii) That requires access to external RAM (> 1 GiB).

Validate the correctness of your code by comparing the output matrix to the reference code.

2.5 Measure or **estimate**, when measuring is not possible, the arithmetic intensity and peak GFlops/s of the reference and your best-optimised implementation of the sparse matrix multiplication on the 3 dataset sizes. **Plot** those values on the x86 and ARM rooflines.

2.6 Port your best implementation to GPU using CUDA. **Identify** and **implement** relevant optimisations to adequately use this device. **Measure** its performance for the 3 dataset sizes.

2.7 Port your best implementation to SYCL. Modify the code as necessary to utilise both CPUs and GPUs simultaneously. **Measure** the scalability of the code by varying the number of cores, CPUs, and GPUs used to process only the largest dataset.

2.8 Use adequate profilers to **identify** and **characterise** the factors limiting the performance of the implementation in 2.7. **Assess** the code and hardware regarding metrics such as resource utilisation, boundness, memory access and usage, and communication. **Identify** possible venues of optimisation.

2.9 Compare the performance and/or scalability of the various implementations. The analysis should focus on the architectural aspects of each computing platform used that affect the code's performance, and how the different platforms suit the code's characteristics. Analyse and characterise the optimised algorithm, but, more importantly, use it to compare the different architectures. **Provide** a critical analysis of the algorithm and hardware performance. Do not forget to compare the architectures in terms of energy footprint (overall, energy to solution, energy per element processed, etc – use your imagination)!

Task 3

Report writing and oral presentation

Write a short essay in English (IEEE double-column format, no longer than 5 pages, including annexes).

Include in this essay:

- (i) Title
- (ii) List of authors
- (iii) Abstract
- (iv) Introduction
- (v) Relevant mid-sections describing the algorithm and its behaviour (computations and memory accesses), but do not include information already provided in the deliverable of Task 1.
- (vi) Conclusions

(vii) References, by order of appearance in the essay, with all data pertinent to find the publication, e.g. author(s), title, place where it was published, who published, and year.

This presentation should only report the activities in Task 2, which includes the rooflines for each partition, but not how they were created – that should be detailed in the deliverable of Task 1.

This final report should be submitted in the BlackBoard form by 23:59 on January 4th.